GUIA TÉCNICO

Classificação Textual e Sentimentos

Propósito — Conteúdo didático autoral — notas técnicas e exemplos independentes. Material de apoio histórico não é reproduzido neste PDF.

Como utilizar — Leia as explicações, execute os exemplos no ambiente indicado e consulte os notebooks correspondentes.

ROTEIRO DO DOCUMENTO

1. Fundamentos e objetivo
2. Modelo e escolhas metodológicas
3. Receitas Python reproduzíveis
4. Interpretação e limitações
5. Exercícios de aplicação
6. Checklist de qualidade

01 | O problema e seu contexto

Classificação Textual e Sentimentos é estudado como técnica computacional, com hipóteses explícitas, dados de demonstração e análise crítica dos resultados.

Cuidados metodológicos

Transformações e vetorizadores devem ficar dentro do Pipeline para evitar vazamento. Observe métricas por classe e erros de negação.

02 | Fundamentos e modelo

Objetivos de aprendizagem

- Construir pipeline supervisionado de texto.
- Avaliar classificação com treino/teste e validação cruzada.
- Compreender ambiguidade de sentimentos, ironia e mudanças de domínio.

Fundamentos conceituais

Em classificação de textos, o vectorizer precisa ser ajustado apenas nos dados de treinamento. A combinação de TF-IDF com regressão logística ou Naive Bayes fornece baselines fortes para coleções pequenas. Uma análise de sentimentos depende de contexto, domínio e rótulos confiáveis; o modelo aprende padrões presentes no conjunto, não o sentimento verdadeiro de qualquer frase.

Modelo matemático e interpretação

Uma função de decisão linear possui score $f(x)=w^{T}x+b$; a regressão logística usa $\sigma(f)=1/(1+e^{-f})$. As métricas precision, recall e F1 complementam acurácia, principalmente com classes desbalanceadas.

Estudo guiado

Pergunta de investigação: como demonstrar o conceito central com um exemplo pequeno, auditável e reproduzível?

Nas células seguintes, inspecione as entradas, execute os procedimentos e explique com suas próprias palavras o resultado. Ao trocar o texto ou o parâmetro, observe se o comportamento permanece coerente. Esses resultados são ilustrações pedagógicas, não benchmarks.

Implementação reproduzível

O código abaixo usa textos sintéticos ou bibliotecas locais. Não executa downloads nem acessa APIs externas. Exemplos opcionais só rodam quando a dependência já está disponível; a ausência deles não interrompe o notebook.

Interpretação guiada: verifique se o resultado corresponde à fórmula e à hipótese discutidas. Anote o que muda com a entrada e quais conclusões não podem ser generalizadas a partir deste exemplo.

Interpretação guiada: verifique se o resultado corresponde à fórmula e à hipótese discutidas. Anote o que muda com a entrada e quais conclusões não podem ser generalizadas a partir deste exemplo.

Exercícios orientados

1. Compreensão: explique qual informação o método preserva e qual descarta. Relacione a resposta com os objetivos do capítulo.
2. Experimento: altere uma frase e um parâmetro do código. Registre a diferença e uma hipótese explicativa.
3. Avaliação crítica: descreva uma situação em que a técnica geraria uma resposta inadequada. Proponha uma mitigação.
4. Transferência: conecte o conceito a uma disciplina anterior e identifique o que não precisa ser revisto.

Resolução comentada (diretriz de resposta): o método representa somente um recorte do fenômeno linguístico; mudanças de entrada alteram a saída segundo a regra estudada. Uma boa análise identifica perda de informação, contexto e risco de generalização. Para a resposta específica, justifique com os valores efetivamente obtidos na execução acima.

Erros comuns, limitações e boas práticas

Este é apenas um corpus sintético pequeno para testar a mecânica da pipeline, não para estimar desempenho generalizável. Para estudos reais, documente origem, licença e divisão de dados.

- Separe demonstração de produção: datasets sintéticos não demonstram generalização.
- Registre versões das dependências e parâmetros; não inclua credenciais no notebook.
- Consulte a documentação oficial antes de adaptar códigos históricos de 2021.

Laboratório adicional — Partição estratificada de texto antes da vetorização

Experimento independente do estado das células anteriores. Use-o como verificação e explique seus limites; resultados em conjuntos sintéticos não são estimativas de desempenho real.

03 | Cookbook Python

Classificação sem vazamento

```
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
X = ['ótimo atendimento','péssimo serviço','muito bom','muito ruim']
y = [1,0,1,0]
modelo = make_pipeline(TfidfVectorizer(),MultinomialNB())
modelo.fit(X,y); print(modelo.predict(['serviço ótimo']))
```

Métricas por classe

```
from sklearn.metrics import classification_report
y_true = [1,0,1,0]; y_pred = [1,0,0,0]
print(classification_report(y_true,y_pred,zero_division=0))
```

04 | Interpretação e limites

Transformações e vetorizadores devem ficar dentro do Pipeline para evitar vazamento. Observe métricas por classe e erros de negação.

05 | Exercícios com respostas orientadoras

1. Altere uma entrada ou parâmetro e descreva o comportamento observado.

Resposta orientadora: mantenha os demais elementos constantes, documente a mudança e explique qual resultado deveria variar pela hipótese escolhida.

2. Cite uma situação em que a técnica falha ou transmite confiança excessiva.

Resposta orientadora: identifique domínio, idioma, quantidade de dados, formato da entrada, possíveis erros sistemáticos e um teste que exponha o limite.

3. Escreva um teste de sanidade que possa ser executado antes de analisar os resultados.

Resposta orientadora: estabeleça forma, tipo, dimensões ou invariantes esperados para os dados, e uma observação que invalidaria a conclusão.

06 | Referências e consulta

Material construído a partir de exercícios autorais da biblioteca, com o conteúdo histórico utilizado apenas como apoio privado. Documentação técnica: Python (docs.python.org), scikit-learn (scikit-learn.org), NLTK (nltk.org), spaCy (spacy.io).

Portfólio GitHub: https://github.com/eduardojr-pereira/data-science-portfolio

Site pessoal: https://eduardo-pereira.webflow.io/